

In-the-Moment Reframing

Framing is easy at a whiteboard. You don't work at a whiteboard. You work in meetings, where someone hands you a vague AI request and a deadline, and the first answer in the room tends to win. These are the moments that come up often enough to name, with a line you can use in each one.

Before you build

"We need this by Friday."

A deadline pushes you to start building before anyone has checked the problem, and then you find out mid-sprint that you're solving the wrong one. Ask for a short framing pause.

"I can start building right now, or I can take 30 minutes to make sure we're building the right thing. Running full speed in the wrong direction won't save us time."

"The CEO wants an agent."

This is a solution handed to you as a requirement, and it gets the least scrutiny because of who asked. Elon Musk's line applies: your requirements are definitely dumb, it doesn't matter who gave them to you. Work back to the outcome before you accept the solution.

"If the agent works perfectly, what changes? What metric moves? What does the CEO actually want to see happen?"

"That's just how our teams are split."

Organizations build systems that mirror their org chart. A company with separate ML and Product teams ships separate ML and Product components even when nothing about the problem needs that split, and the cleanest reframe never gets proposed. Name the constraint out loud.

"Is the solution we're imagining shaped by the problem, or by how our teams are structured?"

"Everyone agrees on the problem."

Usually they agree on the words. The CFO hears cost, the CTO hears feasibility, Legal hears risk, and sometimes nobody's budget actually moves if the project works. A project like that produces interesting findings and no decisions. Find the owner first, then check that everyone means the same thing.

"Whose budget changes if this works? Before we go on, let's each write one sentence: what problem are we solving, who has it, and how will we know we've solved it?"

Compare the sentences before anyone discusses them. The gaps are the disagreements nobody has said out loud yet.

"The demo killed, let's scale it."

Once leadership has seen a demo, questioning the approach starts to feel like questioning the people who championed it. JP Morgan Chase had a working demo and every metric was green, and the question of whether anyone would use it only got asked after launch. Ask it before.

"If this were 100 percent accurate tomorrow, what would still fail?"

If the answer is short, you have a model problem. If it's long, you have a framing problem.

"We have all this data. See what you can find."

This is the science experiment that never finds its problem. The team spends months on interesting patterns, the stakeholders nod at the readout, and nothing changes. Agree on what a useful finding looks like before you start.

"I understand you want to explore. Let's agree on what a successful finding looks like. What decision would you make differently if we found it?"

Once it's running

The harder conversations come later, when the signals say change course and the room doesn't want to hear it. This is where the Signals Canvas pays off, because you're pointing at a signal everyone agreed to instead of delivering bad news.

What you'll hear	What's going on	What to say
"We've already spent \$200K."	Sunk cost	The money is spent either way. The only question is the next dollar.
"It worked when we showed the board."	Demo defense	Show the distribution: the best day versus the average day.
"This was my initiative." (usually unspoken)	Ego protection	Point to the pre-commitment. A fired signal is the process working, and the champion helped define it.
"We don't have time to change course."	Deadline	Three flat iterations cost more time than the pivot.
"Leadership already announced it."	Announcement trap	Ship the narrower, reliable version under the same banner.

Three questions to slow the first answer

When the room has already picked a solution, any one of these from Chapter 1 of the book will slow it down.

	Ask
Inversion	What are the top three ways this could go wrong in production? Be specific.
De-escalation	If I had to solve this without any AI, how would I do it, and would it be good enough?
Feynman Test	Can I explain what this does to a non-expert without using any AI terms?

The book works through every one of these in more depth. It's free at aiframer.dev/book.